

* HAS-A (Association) in Java – Detailed Notes

HAS-A = "uses / contains" relationship

One class **has** an **object** of another class.

1 What is Association?

Association is a relationship between two independent classes where one class uses another.

→ Implemented by creating an **object reference** of one class inside another class.

2 HAS-A Relationship

When a class contains a reference of another class, it is called **HAS-A**.

```
class Car
{
    Engine engine; // Car HAS-A Engine
}
```

3 Why use HAS-A?

- ✓ Code reusability
 - ✓ Loose coupling
 - ✓ Better design (preferred over inheritance)
 - ✓ Easy maintenance
-

4 Types of Association (HAS-A)

◆ 1. Aggregation (Weak Association)

- Objects can exist independently
- Example: **Teacher – Department**

```
class Department {
    String name;
}
```

```
class Teacher {
    Department dept; // Aggregation
}
```

}

✓ Teacher deleted → Department still exists

◆ 2. Composition (Strong Association)

- Objects are dependent
- Example: **House – Room**

```
class Room {  
    int roomNo;  
}
```

```
class House {  
    Room room = new Room(); // Composition  
}
```

✓ House destroyed → Room destroyed

5 Association vs Inheritance

Feature	HAS-A (Association)	IS-A (Inheritance)
Relationship	uses / contains	extends
Coupling	Loose	Tight
Reusability	High	Limited
Keyword	Object reference	extends
Flexibility	More	Less

☞ **Prefer HAS-A over IS-A**

6 Real-World Examples

Class A	HAS-A	Class B
Car	has	Engine
Mobile	has	Battery
Employee	has	Address
Laptop	has	Processor

7 Example with Method Usage

```
class Engine {  
    void start() {  
        System.out.println("Engine started");  
    }  
}  
  
class Car {  
    Engine engine = new Engine();  
  
    void drive() {  
        engine.start();  
        System.out.println("Car is moving");  
    }  
}
```

8 Interview Keywords

- Association
 - Aggregation vs Composition
 - Loose coupling
 - Object dependency
 - Design principle
-

9 One-Line Definition (Exam Ready)

HAS-A relationship represents association where one class contains a reference of another class to use its functionality.

One-to-One Association in Java

Definition

☞ One object of a class is associated with exactly one object of another class.

1 object ↔ 1 object

◆ Real-World Examples

- Person ↔ Passport
 - Employee ↔ ID Card
 - User ↔ Profile
-

◆ Java Example (HAS-A)

```
class Passport {  
    String passportNo;  
}  
  
class Person {  
    Passport passport; // One-to-One  
}
```

◆ Constructor Injection (Best Practice)

```
class Passport {  
    String passportNo;  
  
    Passport(String passportNo)  
    {  
        this.passportNo = passportNo;  
    }  
}  
  
class Person {  
    Passport passport;  
  
    Person(Passport passport)  
    {  
        this.passport = passport;  
    }  
}
```

◆ One-to-One Types

Aggregation

```
class IDCard {}
class Employee
{
    IDCard idCard;
}
```

✓ Independent objects

Composition

```
class Heart {
}
class Human
{
    Heart heart = new Heart();
}
```

✓ Dependent objects

◆ Diagram

Person 1 ----- 1 Passport

◆ Interview One-Liner

One-to-One association links exactly one object of a class with one object of another class using HAS-A.

✓ Complete One-to-One Example

Passport

```
class Passport {
    private String passportNumber;
    private String country;

    Passport(String passportNumber, String country)
    {
        this.passportNumber = passportNumber;
        this.country = country;
    }

    void showPassportDetails()
    {
        System.out.println("Passport No: " + passportNumber);
        System.out.println("Country: " + country);
    }
}
```

Person

```

class Person {
    private String name;
    private Passport passport;

    Person(String name, Passport passport) {
        this.name = name;
        this.passport = passport;
    }

    void showPersonDetails() {
        System.out.println("Name: " + name);
        passport.showPassportDetails();
    }
}

```

Main

```

public class OneToOneDemo {
    public static void main(String[] args) {
        Passport passport = new Passport("M4589231", "India");
        Person person = new Person("Aniket", passport);
        person.showPersonDetails();
    }
}

```

One-to-Many Relationship in Java (HAS-A)

Definition

☞ One object is associated with multiple objects of another class.

1 → many

Example: Department → Employees

Employee

```

class Employee {
    private int empld;
    private String name;

    Employee(int empld, String name)
    {
        this.empld = empld;
        this.name = name;
    }

    void showEmployee()
    {
        System.out.println(empld + " - " + name);
    }
}

```

Department

```
import java.util.*;

class Department {
    private String deptName;
    private List<Employee> employees;

    Department(String deptName, List<Employee> employees)
    {
        this.deptName = deptName;
        this.employees = employees;
    }

    void showDepartmentDetails()
    {
        System.out.println("Department: " + deptName);
        for (Employee emp : employees)
        {
            emp.showEmployee();
        }
    }
}
```

◆ One-to-Many Types

Aggregation

```
class Company {
    List<Employee> employees;
}
```

Composition

```
class Library {
    List<Book> books = new ArrayList<>();
}
```

◆ Diagram

Department 1 ----- * Employee

Interview One-Liner

One-to-Many relationship means one object is associated with multiple objects using a collection in HAS-A relationship.

One-to-Many Relationship in Java (HAS-A) – FULL Example

Definition

☞ One object is associated with multiple objects of another class using HAS-A relationship.

1 → Many

Real-World Example

Department → Employees

- One Department has many Employees
 - Each Employee belongs to one Department
-

Key Concepts Used

- ✓ HAS-A (Association)
 - ✓ One-to-Many
 - ✓ Aggregation
 - ✓ Collection (List)
 - ✓ Constructor Injection
 - ✓ Real-world logic
-

1 Employee Class (Child)

```
class Employee {  
    private int empId;  
    private String name;  
  
    Employee(int empId, String name)  
{  
        this.empId = empId;  
        this.name = name;  
    }  
  
    void showEmployee() {  
        System.out.println(empId + " - " + name);  
    }  
}
```

2 Department Class (Parent – HAS-A Many Employees)

```
import java.util.*;

class Department {
    private String deptName;
    private List<Employee> employees; // One-to-Many

    Department(String deptName)
    {
        this.deptName = deptName;
        this.employees = new ArrayList<>();
    }

    // Add Employee
    void addEmployee(Employee emp)
    {
        employees.add(emp);
    }

    // Remove Employee
    void removeEmployee(Employee emp)
    {
        employees.remove(emp);
    }

    // Display Details
    void showDepartmentDetails()
    {
        System.out.println("\nDepartment: " + deptName);
        for (Employee e : employees) {
            e.showEmployee();
        }
    }
}
```

3 Main Class (Test)

```
public class OneToManyDemo {
    public static void main(String[] args)
    {

        Employee e1 = new Employee(101, "Rahul");
        Employee e2 = new Employee(102, "Aniket");
        Employee e3 = new Employee(103, "Priya");

        Department dept = new Department("IT");

        dept.addEmployee(e1);
        dept.addEmployee(e2);
        dept.addEmployee(e3);

        dept.showDepartmentDetails();

        // Remove one employee
    }
}
```

```
dept.removeEmployee(e2);  
dept.showDepartmentDetails();  
}  
}
```

4 Output

Department: IT
101 - Rahul
102 - Aniket
103 - Priya

Department: IT
101 - Rahul
103 - Priya

5 Why This is One-to-Many?

- ✓ One Department → Many Employees
 - ✓ Uses Collection (List)
 - ✓ HAS-A relationship
 - ✓ Employees exist independently → **Aggregation**
-

6 Diagram (Text)

Department 1 ----- * Employee

7 Aggregation vs Composition (Quick)

Aggregation (Used here)

```
class Company {  
    List<Employee> employees;  
}
```

- ✓ Parent deleted → Child still exists

Composition

```
class House {  
    List<Room> rooms = new ArrayList<>();  
}
```

- ✓ Parent deleted → Child destroyed

👉 Interview One-Liner

One-to-Many relationship means one object is associated with multiple objects of another class using a collection in HAS-A relationship.

Simple Difference (Exam / Interview Ready)

Relationship	How object is stored
One-to-One	Single object reference
One-to-Many	Collection of objects (List / Set)

✓ One-to-One (HAS-A)

👉 No collection

```
class Person {  
    Passport passport; // one object  
}
```

✓ 1 Person → 1 Passport , ✓ Direct object reference

✓ One-to-Many (HAS-A)

👉 Uses collection

```
class Department {  
    List<Employee> employees; // many objects  
}
```

✓ 1 Department → many Employees , ✓ Objects stored in a collection

🔥 One-Line Difference (Perfect Answer)

In One-to-One, a class holds a single object reference, whereas in One-to-Many, a class holds multiple objects using a collection.